FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation Supplementary Material

Anonymous CVPR submission

A. Method and Training Details

A.1. Architecture and Training

On OnTimeBench, each training window holds eight consecutive query positions, each with its own causal history (Table S1). We report the final checkpoint of the seed-0 run.

Table S1. Hyperparameters of FoveaWAM on OnTimeBench and training clusters used across the project.

Parameter	Value
Model	
Input image	224×224 RGB-D, one view
Visual encoder	MAE ViT-B/16, frozen
History length	8 steps (350 ms)
Forecast horizons	8 (50 ms–400 ms)
Summary tokens	16
Layers	12
Width	1024
Heads	16
Feed-forward width	4096
Trainable parameters	271.8M
Action	
Action dimension	7
Chunk length	8
Euler steps	5
Noise samples per query	8
Training	
Optimizer	Fused AdamW
Betas	$(0.9, 0.999)$
Weight decay	10^{-4}
Learning rate	2×10^{-4}
LR schedule	cosine, 5,174 warm-up updates
Batch size	512 windows
Epochs	10 (cap: 20)
Reported checkpoint	25,870 updates
Gradient clipping	0.1
$\lambda_F, \lambda_P, \lambda_R, \lambda_{\text{aux}}$	0.1
Training precision	BF16 compute, FP32 parameters
Training clusters	$64 \times$ RTX PRO 6000, $32 \times$ H100

Scene tokens. Each patch token of the frozen MAE ViT-B/16 [5] adds a learned view embedding and linear embeddings of its mean valid metric depth and of the base-frame position $\mathbf{x}_i$ of its back-projected center pixel; a patch without valid depth at its center receives no position embedding. Only the patch queries and keys receive 3D rotary embeddings from $\mathbf{x}_i$, and the same points are the source points of the sparse 3D flow.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities and accelerations from finite differences over the elapsed times, and slot 7 holds the current pose in the base frame. Each dimension is standardized with training statistics, invalid dimensions are zeroed, and three validity flags (pose, interval, acceleration) are appended; a slot without any valid feature is masked out of attention. The robot tokens hold the end-effector position, rotation vector, and gripper opening at the same offsets, and 16 learned embeddings form the summary tokens.

Heads and losses. Each head is a linear layer on the final world-stream states, except the target reconstruction, a two-layer MLP. The pose head reads the mean summary state ($\mathcal{L}_P$, Eq. 3). The flow head gives each patch point's displacement and visibility logit at every horizon ($\mathcal{L}_F$, Eq. 2, with scale $s = 0.715$ m and visibility weight 1); valid but invisible points still supervise their displacement. Flow labels follow the rendered surface point at each patch center through the ground-truth rigid motion of its MuJoCo geometry; a future point is visible if it projects into the image onto the same geometry within a depth tolerance of $\max(1\,\text{mm}, 0.001z)$. $\mathcal{L}_{\text{aux}}$ sums four losses: squared errors on future metric depth (valid pixels), on the RGB of surfaces that move by more than 1 mm, and on layer-normalized DINOv3 features [12], and an absolute error on the current target's RGB region reconstructed from the mean summary state.

Action stream. Each action token embeds a seven-dimensional command: a base-frame translation increment, a left-relative rotation vector, and a gripper command, relative to the measured end-effector pose at application. Command j of query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution

(Eq. (S1)), and commands are standardized over the training split. At deployment, the gripper changes state only when its command leaves a deadband of 0.5 in Ball capture and 0.1 in every other task, including Ball capture, table.

A.2. Pretraining on DROID and Fine-Tuning

The real-robot base trains on DROID [8] (15 Hz) with the first exterior camera and its depth, using the 3D point tracks and calibrations released by PointWorld [6]. Its 42,087 usable episodes yield 598,889 anchor windows, 60,149 of them for validation; each anchor expands into 8 query positions as on OnTimeBench, and only the anchor carries flow and DINOv3 labels. Each patch follows one PointWorld track that starts inside it, preferring the one with the largest consistent motion so that small moving objects are covered; 47% of the point–horizon slots carry a label.

DROID provides no object poses, so the base omits the target-state tokens and pose head. The losses for actions, sparse 3D flow, and future DINOv3 features have weights 1, 0.1, and 0.1, respectively. The flow loss has no visibility term. We cap pretraining at 20 epochs (21,060 optimizer updates) with AdamW and a global batch size of 512. The backbone uses FP32 and the frozen visual encoder uses BF16.

Each real-robot policy starts from a DROID-pretrained base and is fine-tuned on its task's demonstrations, with target-state tokens and new prediction heads initialized from scratch. The fine-tuning budget is 20,000 optimizer updates per task with a global batch of 32. AdamW uses 500 warm-up updates followed by cosine decay, with peak learning rates of 3×10^{-5} and 2×10^{-5} for the 12- and 24-layer bases, respectively.

A.3. Figure Details

In Fig. 1(a), the real-robot scene is reconstructed from RGB with MoGe-2 [14], and the overlaid SE(3) futures and sparse 3D flow are illustrative. The overlays of Fig. 2 come from one recorded OnTimeBench scene: flow samples with a displacement of at most 2 mm are drawn as dots, and the target axes are drawn at 0 and 400 ms. In Fig. 3, the thumbnails are Blender renderings of a recorded rollout with the flow enlarged for display; token counts, layer stacking, and action icons are schematic. Fig. 4(c) renders expert rollouts with past target poses and motion cues.

B. OnTimeBench and Evaluation Protocol

B.1. Benchmark Design

The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [13, 20] with a Franka Panda arm and a parallel gripper under robosuite's operational-space pose controller, with a control period of 50 ms and a physics step of 2 ms. A fixed third-person camera with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector translation and rotation-vector increment in the base frame with a gripper command, clipped to 0.10 m per translation axis and 0.5 rad per rotation component. Table S2 lists the fourteen tasks.

B.2. Scenarios, Demonstrations, and Splits

Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges. Table S2 summarizes target motion. Other sampled parameters include harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, and ramp slopes of 5–15°. In an occluded scene, the target is hidden for 0.5–3.0 s and then reappears. Only physically feasible scenes are kept, and evaluation replays the stored scenes.

The expert is a task-specific closed-loop state machine with privileged simulator state that anticipates the target's motion, for example by predicting the interception point in the capture tasks. A demonstration is kept if the expert succeeds, a replay of its actions reproduces the episode, and its path and jerk pass quality checks. The training pool holds 11,200 demonstrations, 800 per task, recorded at 20 Hz with RGB-D, robot states, commands, and simulator ground truth, and 350 further episodes form the validation split; policies receive only the inputs of Sec. 3.

We select configurations on 1,400 development scenes (100 per task) and report all OnTimeBench results on 1,400 held-out scenes shared by all methods, generated with the same ranges from seeds used in no training, validation, or development episode.

B.3. Real-Time Evaluation and Metrics

Executor. All methods share one executor on the 50 ms control clock. Command j of query t is meant for $\tau_t + j\Delta_a$, and its execution time (Sec. 3.1) is

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}, \tag{S1}$$

where $d_{\mathrm{task}} = 50$ ms for the post-impact grasp, which applies each command 50 ms after it is issued, and $d_{\mathrm{task}} = 0$ otherwise. As in RTC [2], latency counts in whole control periods: a query with total latency L_t hands over its chunk at the first boundary at or after it ends, $k_t = \lceil L_t/\Delta_a \rceil$ periods after capture, and execution continues from command k_t, the one meant for that boundary. The policy then observes again and starts the next query; meanwhile the previous chunk continues, so a policy replans as often as its latency allows. A chunk holds the method's native number H_m of commands (Table S3) and is dropped if $k_t \geq H_m$. Before the first chunk arrives and whenever a chunk runs out, the controller holds the current arm target and gripper command; no command is repeated. The executor runs asynchronously, as in the streaming of DynamicVLA; no method, including FoveaWAM, inpaints across chunks as in RTC.

Table S2. **The fourteen OnTimeBench tasks.** The target is the body whose pose the policy receives (Sec. 3.1). For placement it is the receptacle. For passage it is the wire entry, and the rest of the wire is context. Motion gives linear speed in m/s or the wire's angular speed in degrees per second. For each capture task, we estimate speed from the two samples 50 ms apart that bracket the first recorded ramp exit in each of 100 held-out fixed-robot reference trajectories. A grasp requires bilateral fingerpad contact, a lift of at least 0.05 m, and a continuous 0.5 s hold. An episode fails at unsafe contact, leaving the workspace, a drop, or the time limit. In the group column (Table 3), R marks tasks where the target rotates during the lead and its orientation matters at contact. C marks tasks with an obstacle clearance or a driver such as a conveyor, ramp, or striker.

Task	Code	Target	Motion range	Success	Limit (s)	Group
Harmonic grasp	`grasp`	block	0.03–0.20	grasp	8	R
Conveyor grasp	`t1_belt`	part	0.04–0.10	grasp	8	C
Translating grasp	`t1_xyz`	part	0.04–0.15	grasp	8	–
Rotating grasp	`t1_so3`	part	0.04–0.15	grasp	8	R
Occluded grasp	`t1_occ`	part	0.04–0.10	grasp without canopy contact	8	C
Moving receptacle	`t2_trans`	receptacle	0.03–0.06	held object at rest inside	10	–
Rotating receptacle	`t2_se3`	T-shaped receptacle	0.03–0.06	held T-part at rest inside	10	R
Wire passage	`t3_phase`	wire entry	2.5–7.5°/s	align within 30 mm, pass without contact	21	R, C
Full wire passage	`t3_full`	wire entry	2.5–7.5°/s	grasp wand, pass, exit, release	32	R, C
Ball capture	`t4_sphere`	ball	0.67–1.14	grasp before the tray	6	C
Ball capture on table	`t4_sphere_tabletop`	ball	0.59–1.03	grasp while rolling	6	C
Slider capture	`t4_slider`	slider	0.55–1.12	grasp while sliding	6	C
Can capture	`t4_can`	can	0.59–0.97	grasp before tipping	6	R, C
Post-impact grasp	`t5_replan`	part	0.12–0.14	grasp along its axis	6	R, C

Protocols. In real time, physics advances during each query by the method's latency, drawn per query from its distribution measured on one RTX 5090, while the rollouts themselves run on RTX PRO 6000. With paused physics, inference takes no physics time and a policy replans every 50 ms; this removes the wait for inference but also raises the feedback rate, so it is a reference rather than an upper bound. The latency sweep adds 0 to 200 ms in steps of 50 ms after inference in real time.

Latency measurement. We time each method alone on one RTX 5090 at batch size 1 in each baseline's release precision and in compiled BF16 for FoveaWAM, over three rounds of 30 warm-up and 300 measured calls on recorded observations with the required causal history. A timed query starts from the newest observation and ends when its first command is available. It includes all computation that needs the newest frame, such as preprocessing, image encoding, the online flow and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. PUMA is timed with its official three-camera input and the other baselines with two cameras. FoveaWAM runs five Euler steps with TF32 disabled; compilation and warm-up on recorded observations finish before physics starts, after which the history and the inference seed are reset. The forecast heads of FoveaWAM are skipped at deployment and excluded from time-to-action. A query misses its deadline when this time-to-action exceeds 50 ms, and the action age of a command is its execution time minus the capture time. Appendix C.2 reports the measured values.

Metrics. SR weights the 14 tasks equally. The position error at first contact e_p is the distance of the end effector from the expert's contact frame on the target; in the placement tasks, which start with the object held, it uses the contact of the held object with the receptacle and the expert's placement pose. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact remain in the SR denominator. Paired 95% intervals bootstrap held-out scenes within each task, jointly across methods.

B.4. Baselines

Table S3. **Baseline training.** The training budget gives optimizer updates and global batch size. H is the number of commands returned per query. The parameter counts include trainable parameters only. DOM ckpt. denotes the released DynamicVLA checkpoint trained on DOM.

Method	Initialization	Updates×batch	H	Params
ACT [19]	scratch	50,000 × 32	100	83.9M
DP [3]	scratch	50,000 × 32	15	288.7M
$\pi_{0.5}$ [1]	`pi05_base`	30,000 × 256	10	3,353.4M
$\pi_{0.5}$+SE(3)	`pi05_base`	30,000 × 256	10	3,357.7M
DynamicVLA [18]	DOM ckpt.	50,000 × 44	20	99.1M
PUMA [4]	official	40,000 × 64	16	4,516.5M
OpenWAM-α [15]	official	34,490 × 192	32	6,021.2M
DynamicWAM [10]	official	40,000 × 128	16	988.8M
DynamicWAM+SE(3)	official	40,000 × 128	16	989.4M
FoveaWAM	scratch	25,870 × 512	8	271.8M

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe

(Table S3), trains on the same 11,200 demonstrations mapped to its action space through the same Cartesian controller, and is configured on development scenes. Rollouts use the agent view each baseline was trained with, any wrist slot masked, and the histories of Table 1; masking the wrist view changes p95 latency by at most 3.8 ms in the seven configurations timed both ways. DP uses its authors' real-robot DDIM sampler with 16 steps, and OpenWAM-α runs with video decoding disabled and is timed with and without its official cache and compilation.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags. A two-layer MLP with a zero-initialized output layer maps them to the model's width; for $\pi_{0.5}$ they follow the image tokens in the prefix, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. Training follows the base model's recipe.

C. Additional Simulation Results

C.1. Per-Task Results

Table S4 uses the Native real-time protocol with +0 ms extra delay. FoveaWAM reaches 93% on moving-receptacle placement and 66% on rotating-receptacle placement, compared with 63% and 11% for $\pi_{0.5}$+SE(3). On full wire passage, FoveaWAM reaches 40% versus 12% for DynamicVLA. It ties the strongest baseline on phase-only wire passage, tabletop ball capture and can capture. The largest recorded failure classes for FoveaWAM are rim impulse violations (160 of 1,400 scenes) and wire contacts (55 scenes).

C.2. Latency

Table S5 lists all eleven configurations at five added latencies; Fig. 5 plots the points from 0 to 150 ms. At +200 ms the 8-command chunk of FoveaWAM and the 10-command chunk of $\pi_{0.5}$ no longer cover the interval until the next chunk arrives: three and four commands remain for five and six control periods, and the controller holds its last target in between. That column therefore measures chunk length rather than the response to delay.

Table S5. **Success versus added latency** on 1,400 held-out OnTimeBench scenes per point (SR in %, real-time protocol). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1. Blue/bold and light blue mark the best and second-best values per column.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	41.6	43.9	36.8	24.6	16.6
Diffusion Policy	31.9	26.6	21.1	16.1	14.4
$\pi_{0.5}$	54.4	47.6	40.1	28.1	13.6
$\pi_{0.5}$ + SE(3)	58.7	41.9	32.6	28.0	14.9
DynamicVLA	51.1	46.8	30.9	23.5	18.0
PUMA	61.4	53.2	36.4	31.1	**27.2**
OpenWAM-α	46.1	36.2	28.0	24.8	22.1
OpenWAM-α (off)	15.8	13.7	14.3	10.4	9.5
DynamicWAM	41.1	27.8	25.0	20.6	21.0
DynamicWAM + SE(3)	41.7	32.8	27.8	24.9	22.9
FoveaWAM	**82.1**	**72.5**	**53.7**	**34.1**	12.8

Timing. Table S6 reports replay timings by stage for the development checkpoint at 20,696 updates.

Table S6. **Latency breakdown** of one query of compiled BF16 FoveaWAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot. The policy total uses 900 measured queries and excludes pose tracking; magenta dashes mark pending measurements.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Observation encoding + H2D	4.83	5.28
World stream (once per query)	3.93	4.22
Action stream (5 Euler steps)	4.27	4.41
Policy time-to-action (total)	14.66	15.65
Training-only readouts (for reference)	0.37	0.41
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

C.3. Results on DOMINO

We use the 24 single-arm Franka tasks of the clean level-1 setting of DOMINO [4] with its demonstrations, scenes, and native evaluator. The manipulation score (MS) scales route completion by penalties for leaving the workspace or view and for collisions. FoveaWAM reads the ground-truth pose of the instructed target from the simulator and masks slots without a valid pose (Appendix A.1). The RGB-based baselines do not receive this pose.

Data and adaptation. The data split has 1,200 training and 240 validation demonstrations. Policies execute absolute joint-position targets with DOMINO's native controller. Their state and action projections use its eight-dimensional joint space, with normalization fitted to the training data. The official sampler of DynamicWAM skips demonstrations too short for its windows, which leaves it 1,054 training and

Table S4. **Per-task held-out success** (%) on OnTimeBench for every method of Table 1, Native real-time protocol: each method's measured inference latency plus **+0 ms extra delay**, with physics continuing during inference; 100 paired scenes per task. FoveaWAM uses compiled BF16 C (p95 15.1 ms). DP: Diffusion Policy. DVLA: DynamicVLA. OWAM: OpenWAM-α with official cache/compile; OW-off: both disabled. DWAM: DynamicWAM. +: with our SE(3) history tokens. Δ: FoveaWAM minus the strongest other method on that task; ties follow table order. The task-balanced row compares against the strongest overall baseline, PUMA. Brackets give pointwise 95% paired scene-bootstrap intervals, conditional on the selected comparator and checkpoint (100,000 resamples; no selection adjustment). The last two rows give first-contact position error e_p (mm), averaged over the 13 contact tasks, and the number n of episodes with a contact (Appendix B.3); –: no complete 13-task mean. Δ shading is blue for higher SR and orange for lower, saturating at 15 points. Blue/bold and light blue mark the best and second-best displayed values.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	OWAM	OW-off	DWAM	DWAM+	**Ours**	Δ [95% CI]
Harmonic grasp	49.0	31.0	51.0	84.0	39.0	85.0	41.0	1.0	50.0	47.0	**99.0**	+14.0 [7.0, 22.0]
Conveyor grasp	45.0	45.0	96.0	62.0	89.0	89.0	62.0	0.0	59.0	60.0	**100.0**	+4.0 [1.0, 8.0]
Translating grasp	14.0	33.0	59.0	42.0	61.0	51.0	53.0	0.0	53.0	18.0	**77.0**	+16.0 [4.0, 28.0]
Rotating grasp	24.0	20.0	32.0	53.0	26.0	67.0	42.0	3.0	30.0	35.0	**71.0**	+4.0 [-8.0, 16.0]
Occluded grasp	30.0	28.0	37.0	60.0	80.0	84.0	34.0	0.0	44.0	38.0	**100.0**	+16.0 [9.0, 23.0]
Moving receptacle	26.0	24.0	34.0	63.0	41.0	24.0	27.0	0.0	8.0	6.0	**93.0**	+30.0 [21.0, 40.0]
Rotating receptacle	0.0	0.0	1.0	11.0	1.0	0.0	0.0	0.0	0.0	0.0	**66.0**	+55.0 [45.0, 65.0]
Wire passage	99.0	99.0	**100.0**	89.0	99.0	87.0	50.0	58.0	23.0	78.0	**100.0**	+0.0 [0.0, 0.0]
Wire passage, full	0.0	3.0	1.0	0.0	12.0	0.0	0.0	0.0	0.0	0.0	**40.0**	+28.0 [16.0, 40.0]
Ball capture	36.0	16.0	**46.0**	29.0	41.0	39.0	45.0	13.0	24.0	20.0	38.0	-8.0 [-18.0, 2.0]
Ball capture, table	71.0	81.0	**100.0**	**100.0**	88.0	**100.0**	99.0	88.0	98.0	**100.0**	**100.0**	+0.0 [0.0, 0.0]
Slider capture	56.0	30.0	58.0	62.0	28.0	76.0	71.0	31.0	47.0	63.0	**79.0**	+3.0 [-5.0, 12.0]
Can capture	91.0	31.0	98.0	99.0	72.0	**100.0**	**100.0**	5.0	96.0	**100.0**	**100.0**	+0.0 [0.0, 0.0]
Post-impact grasp	42.0	5.0	49.0	68.0	39.0	57.0	21.0	22.0	43.0	19.0	**87.0**	+19.0 [7.0, 31.0]
Task-balanced	41.6	31.9	54.4	58.7	51.1	61.4	46.1	15.8	41.1	41.7	**82.1**	+20.8 [18.4, 23.1]
e_p (mm)	32.2	36.1	31.3	34.8	30.0	27.2	31.8	–	31.2	36.1	**22.0**	–
n	1110	946	1237	1191	1204	1272	1115	493	1193	1029	1285	–

210 validation demonstrations and none for two tasks; all 24 tasks remain in the test set. DynamicWAM trains for 40,000 updates at global batch 128. $\pi_{0.5}$ starts from its official base, trains for 30,000 updates at global batch 256, and is evaluated with its final EMA. PUMA keeps its published weights and frozen teacher and trains at global batch 64 with early stopping on validation action MSE, which selected update 6,000. For development, FoveaWAM trains from scratch with a global batch of 512 and stops after 1,575 updates. The lowest validation loss selects the checkpoint at update 1,365. Development rollouts select three executed actions per query.

C.4. Ablation Controls

The six controls of Sec. 5.3 (Table 3) share the data, training recipe, and executor. Each is evaluated at the final checkpoint of its run. Removing a loss also removes its head. The control without target history removes slots 0–6 and keeps slot 7, the current pose (Appendix A.1). The uniform fine flow control removes $\mathcal{L}_P$ and the pose head. It supervises flow at 4×4 points in each patch, forming a 56×56 grid with labels from the same ground-truth rigid motion. Table S7 reports the controls in all three settings, including paused physics.

Table S7. **Ablation controls in all three settings.** Task-balanced SR (%) of FoveaWAM on held-out OnTimeBench scenes and its change for each control (Δ, points). The Native, +150 ms, Rot., and Ctx. columns equal Table 3; Paused adds the paused-physics protocol. Red marks drops and green marks gains, with shading scaled within each column. Rot./Ctx.: rotation-sensitive and context-active tasks (Table S2).

	Success rate, all tasks			Groups (Native)	
	Paused	Native	+150 ms	Rot.	Ctx.
FoveaWAM	**90.3**	**82.1**	**34.1**	**80.4**	**82.7**
Precision allocation					
Uniform coarse flow	+4.4	−4.5	−0.5	−8.6	−4.6
Uniform fine flow	+1.3	+0.8	−0.9	−3.4	−2.6
w/o flow	+3.1	−4.2	−1.4	−11.7	−3.9
Target motion history					
w/o history	−19.5	−19.6	+0.2	−24.0	−2.3
Future supervision					
w/o future	+0.2	−4.3	−3.4	−12.6	−2.1
w/o $\mathcal{L}_{\text{aux}}$	−3.8	−2.6	−1.9	−11.6	+0.1

D. Real-Robot Experiments

D.1. Platform and Tasks

The robot is a Flexiv Rizon 4s arm with a GN01 gripper. Policies send absolute Cartesian tool-center-point poses and

gripper targets at 20 Hz. Demonstration collection logs robot control at 100 Hz. The setup uses two RealSense D435 cameras, one fixed and one mounted on the wrist, both recording RGB at 640×480 and 60 Hz. We calibrate the fixed camera to the robot base with a ChArUco board. In the planned evaluation, FoveaWAM uses RGB-D frames and target poses from the fixed camera. Baselines use RGB from both cameras. The task devices are installed on this platform one at a time (Fig. 6; Table S8).

Table S8. **Real-robot tasks.** A grasp succeeds when the robot holds the target stably and lifts it clear of its support. The time limit is 30 s for every task.

Task	Target motion	Success
Conveyor Grasp	Block on a conveyor belt	Grasp
Moving Box Placement	Open box on a conveyor belt	Block stays in box
Turntable Grasp	Rectangular block on a turntable	Grasp across short sides
Toy Car Grasp	Toy car moving across the table	Grasp
Tennis Ball Interception	Tennis ball rolls with randomized initial speed and heading offset	Stop or deflect with the gripper
Ramp Cart Grasp	Block carried by a cart on an incline	Grasp

D.2. Pose Tracking and Labels

Offline labels. We label the recorded fixed view with masks from Grounding DINO [9] and SAM 2.1 [11]. FoundationPose [16] tracks the metric object mesh forward and backward using depth from FoundationStereo [17]. We check agreement between directions and use tool-center-point kinematics to fill occluded frames during rigid grasps. We retain 595 demonstrations with valid target poses on 97.1% of their frames. Tennis-ball labels use translation only. For sparse scene flow, we lift CoTracker3 [7] tracks to the robot base frame using stereo depth. Future slots are sampled at the 50–400 ms prediction horizons by interpolating valid observations.

Online pose estimation. We estimate target poses from the fixed camera's aligned RGB-D frames and transform them to the robot base frame using the camera-to-base calibration. The policy samples these estimates at 20 Hz to build its target-state history.

D.3. Training and Evaluation Protocol

The shared corpus contains 600 teleoperated demonstrations, with 101 for Turntable Grasp, 99 for Tennis Ball Interception, and 100 for each other task. ACT trains one policy per task for 50,000 updates. DynamicWAM fine-tunes one policy across the six tasks for 20,000 updates. Its deployment configuration uses asynchronous 16-action chunks, requests a new chunk with four commands (200 ms) remaining, and blends overlapping commands. Appendix A.2 describes fine-tuning for FoveaWAM.

We run 20 trials for each task and method in separate groups. An operator resets the scene before each trial and records its outcome. We count timeouts, operator interventions, and safety stops as failures. Table S6 reports robot timing.

D.4. Results and Failure Analysis

Table S9. **Real-robot success with intervals.** Success rates (%) pooled over the trials of each method, with 95% Wilson intervals. Table 4 gives per-task rates.

Method	All [95% CI]
$\pi_{0.5}$	—
DynamicWAM	—
ACT	—
FoveaWAM	—

Table S9 gives pooled success rates and intervals. Figure S1 groups failures by observable events, including late or misaligned grasps, slips or drops, collisions or safety stops, timeouts, and loss of target tracking. [The dominant failures by method and the evidence for a timing error.]

Figure placeholder: Real-robot failures

(a) Failures by category. One stacked bar per method. **(b) Representative trials.** Show unretouched fixed-view frames and mark the events that determine the outcomes.

Figure S1. **Real-robot failures.** (a) Failure counts by category and method. (b) Frames from representative trials. [Tasks, methods, and outcomes of the shown trials.]

References

[1] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 3

[2] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2

[3] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 3

[4] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 3, 4

[5] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 1

[6] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 2

[7] Nikita Karaev, Iurii Makarov, Jianyuan Wang, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker3: Simpler and better point tracking by pseudo-labelling real videos. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. 6

[8] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[9] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 6

[10] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 3

[11] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, pages 28085–28128, 2025. 6

[12] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 1

[13] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 2

[14] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. In *Advances in Neural Information Processing Systems*, pages 35928–35959, 2025. 2

[15] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 3

[16] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 6

[17] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. FoundationStereo: Zero-shot stereo matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5249–5260, 2025. 6

[18] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 3

[19] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 3

[20] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri,

Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 2